



Temporal Multi-Features Representation Learning-Based Clustering for Time-Series Data

(IEEE 2024; JAEHOON LEE, DOHEE KIM, SUNGHYUN SIM)

MinGyun Kang

alsrbs3345@pusan.ac.kr

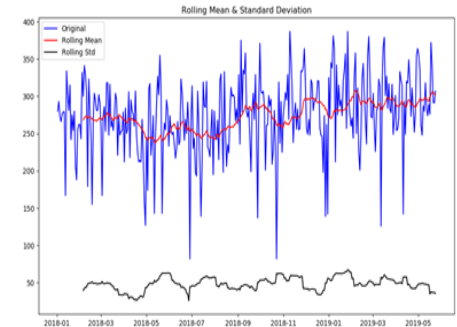
Contents

- 1. Background
- 2. Temporal Multi-Features Representation Learning-Based Clustering for Time-Series Data (TMRC)
 - (IEEE 2024; JAEHOON LEE, DOHEE KIM, SUNGHYUN SIM)
- 3. Deep time-series clustering via latent representation alignment (DEETO)
 - (Sangho Lee, Chihyeon Choi, Youngdoo Son)
- 4. Comparison

Background

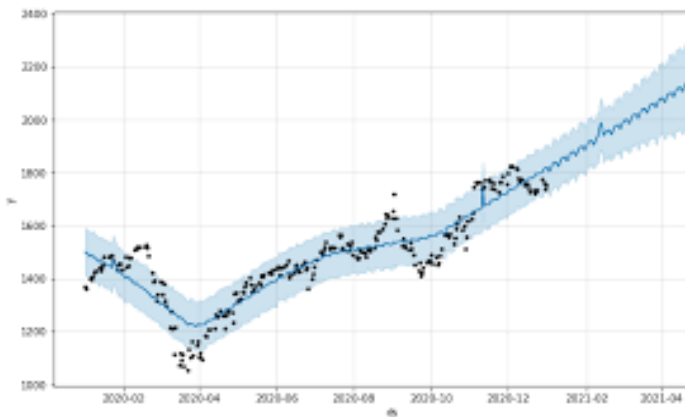
■ What is Time-Series Data?

- 시계열 데이터란 일정 시간 간격으로 기록된 데이터의 연속을 의미
 - Forecasting (예측): 주가, 날씨, 발전량, 수요 예측 등
 - Classification (분류): 정상/고장 신호 분류 등
 - Clustering (클러스터링): 유사한 패턴 그룹화, 이상치 탐지 등
 - ✓ 데이터의 빠른 탐색과 패턴 식별을 가능하게 하는 핵심 기술



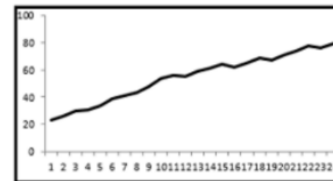
■ Characteristics of the time-series

- 비정상성, 비선형성, 주기성, 계절성과 같은 다양한 특성을 가짐
- 데이터 자체가 매우 다양한 특성을 동시에 가지기 때문에 데이터의 복잡성이 높고, 이는 각 데이터마다 고유한 왜곡을 발생시킴

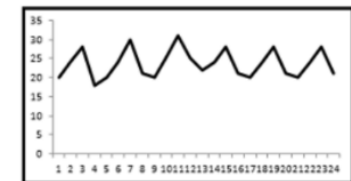


Forecasting

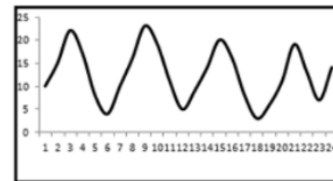
Components of Time Series



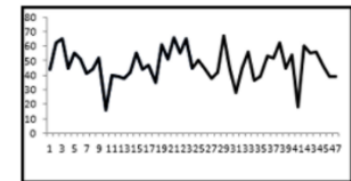
(a) Trend



(b) Seasonality



(c) Cyclicality

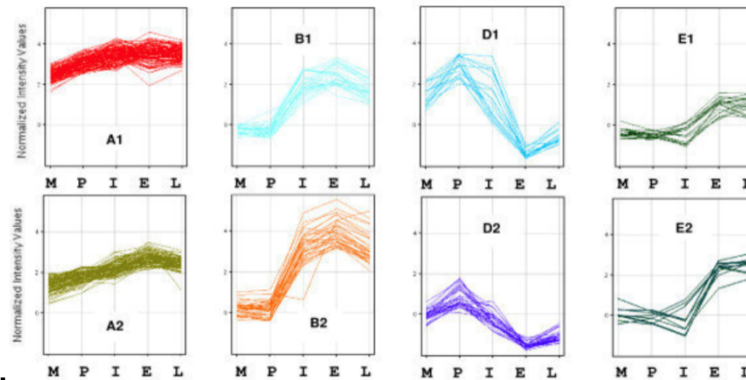


(d) Irregular

Background

■ Clustering

- 비지도 학습으로 레이블이 없는 상태에서 데이터의 숨은 구조나 패턴을 파악하기 용이
- 방대한 데이터 속에서 유사한 패턴을 묶어내고 이상치를 탐지할 수 있어 데이터 마이닝에서 핵심 기술로 꼽힘



■ Three approaches exist for time-series clustering

- 1) Similarity-based (유사도 기반)
 - 유클리드 거리나 DTW 같은 거리 측정 알고리즘을 사용해 시계열 데이터 간의 거리나 유사성을 직접 측정하여 클러스터링하는 가장 직관적인 방법
- 2) Feature-extraction-based (특징 추출 기반)
 - 고차원의 시계열 원본 데이터에서 핵심 특징만을 뽑아 저차원 벡터로 변환한 뒤 클러스터링
- 3) Representation-learning-based (표현 학습 기반)
 - 딥러닝 모델을 사용하여 고차원 데이터를 저차원의 잠재 변수로 변환하여 학습한 뒤 클러스터링

Background

■ Representation-learning-based clustering (표현 학습 기반)

- 현재 가장 우수한 성능을 보여주고 있는 접근 방법
- 심층 신경망을 사용하여 고차원의 시계열 데이터를 저차원의 잠재 변수로 변환하는 방식
 - 이 과정에서 노이즈나 불필요한 정보는 버리고, 데이터의 내재적인 구조와 비선형적 특성만을 남김
- 주요 알고리즘
 - 1. Deep embedded clustering (DEC): 오토인코더로 특징을 추출하고, 동시에 클러스터링을 최적화하는 모델
 - 2. Deep temporal clustering (DTC): 시계열 특화 오토인코더와 시간적 클러스터링 레이어를 결합한 모델
 - 3. Deep temporal clustering with representation learning (DTCR): Seq2Seq 구조를 활용하여 재구성과 클러스터링을 동시에 수행

■ 기존 표현 학습 모델의 한계점

- 1. 다양한 시간적 패턴의 부재
 - 기존 모델들은 시계열 데이터를 하나의 덩어리로 보고 학습하기 때문에, 데이터 내부에 혼재된 다양한 시간적 패턴을 세밀하게 고려하지 못함. 저주파(추세)와 고주파(노이즈, 급격한 변화)가 섞여 있는데, 이를 뭉뚱그려 처리함
- 2. 특정 패턴에 편향된 학습
 - 고차원 데이터에 내재된 다양한 패턴이 제대로 반영되지 않으면, 모델은 가장 두드러지는 특정 패턴만을 편향적으로 학습함. 이는 특징 추출의 정확도를 떨어뜨리고, 최종 클러스터링 품질을 저하시킴

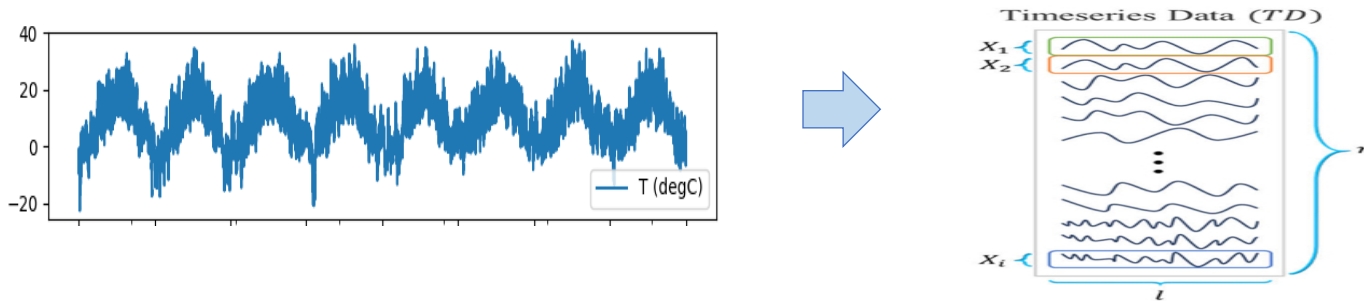
Temporal representation-learning-based clustering(TMRC)

- Temporal representation-learning-based clustering(TMRC)
 - 복잡한 시계열 데이터를 뭉뚱그려 학습하지 않고, 분리 → 학습 → 통합하는 구조
 - 이를 통해 데이터에 내재된 다양한 패턴을 놓치지 않고 포착하는 것을 목표로 함
 - TMRL (Learning): 다양한 시간적 패턴을 포착하기 위한 새로운 표현 학습 방법
 - TMRC (Clustering): TMRL을 활용하여 클러스터링의 성능을 극대화하는 프레임워크
- Step 1: Generate Temporal Multi-features (Decomposition)
 - VMD를 통해 각 시계열을 서로 다른 주파수 대역을 갖는 k개의 모드로 분해
- Step 2: Representataion (TMRL)
 - 생성된 k개의 feature set 각각에 대해 k개의 LSTM-Autoencoder를 독립적으로 학습
- Step 3: Clustering (Ensemble)
 - 각 Autoencoder에서 추출된 잠재 변수들을 하나로 통합
 - 다양한 시간적 특성이 반영된 통합 표현 Z를 기반으로 거리 기반 클러스터링(K-Means) 수행

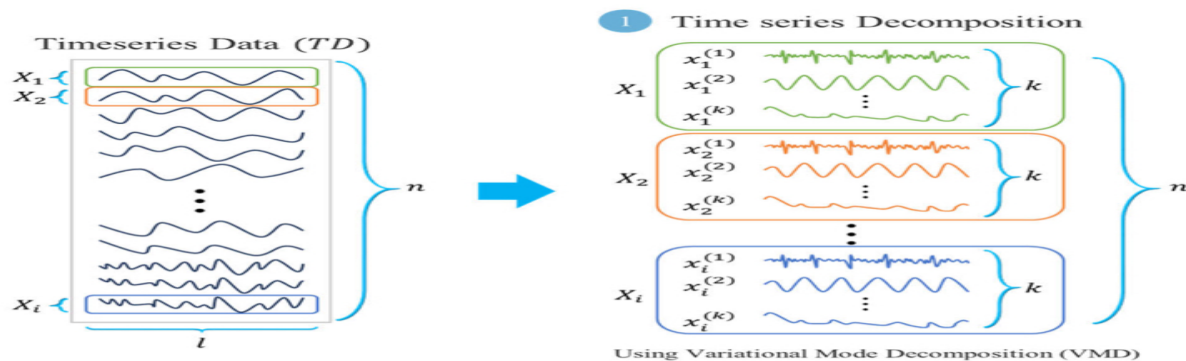
Temporal representation-learning-based clustering(TMRC)

■ Step 1) Generate temporal multiple-features

- 1-1) 전체 길이 T 를 갖는 원본 시계열 데이터에 윈도우 슬라이싱(Window Slicing) 기법을 적용하여, 고정된 길이 ℓ 을 갖는 다수의 부분 시계열(Sub-sequence)로 분할 ($\ell = 10, 50, 100$)
 - 딥러닝 모델은 학습을 위해 많은 데이터를 필요로 하는데, 자르기 전엔 전체 길이 T 짜리 데이터 1개 뿐임
 - 이걸로는 학습이 불가능하기에 윈도우 슬라이싱을 통해 길이 ℓ 짜리 데이터로 수백, 수천 개 늘리는 작업을 거침



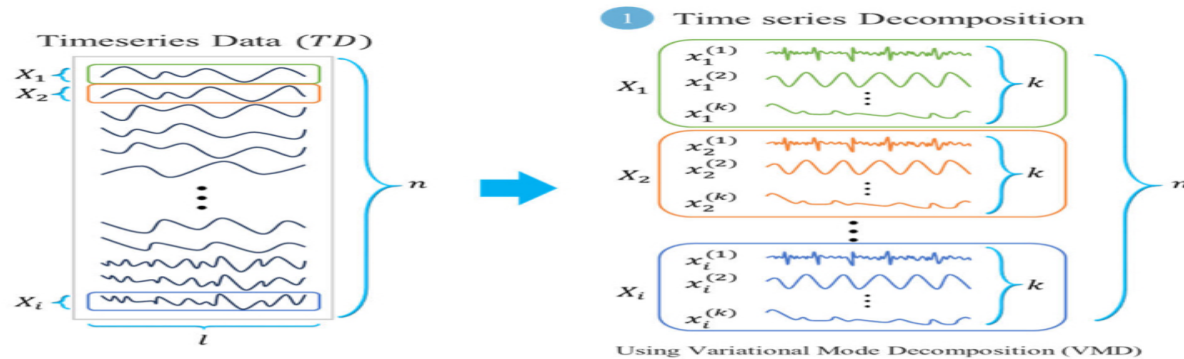
- 1-2) 길이가 ℓ 인 각 부분 시계열(Sub-sequence) $X_1, X_2, \dots, X_i$ 에 대하여, 사용자가 사전에 정의한 모드 개수 k 를 적용해 개별적으로 VMD를 수행 ($k = 2, 3, 4, 5, 6$)



Temporal representation-learning-based clustering(TMRC)

■ Step 1) Generate temporal multiple-features

- 1-2) 길이가 ℓ 인 각 부분 시계열(Sub-sequence) $X_1, X_2, \dots, X_i$ 에 대하여, 사용자가 사전에 정의한 모드 개수 k 를 적용해 개별적으로 VMD를 수행 ($k = 2, 3, 4, 5, 6$)



■ Variational Mode Decomposition(VMD)

- 입력된 시계열 데이터를 주파수 영역에서 분석하여, 각기 다른 중심 주파수를 갖는 k 개의 고유한 모드들로 분리해내는 기법
- 중심 주파수 u_k 를 기준으로 낮은 주파수부터 높은 주파수를 순서대로 분해

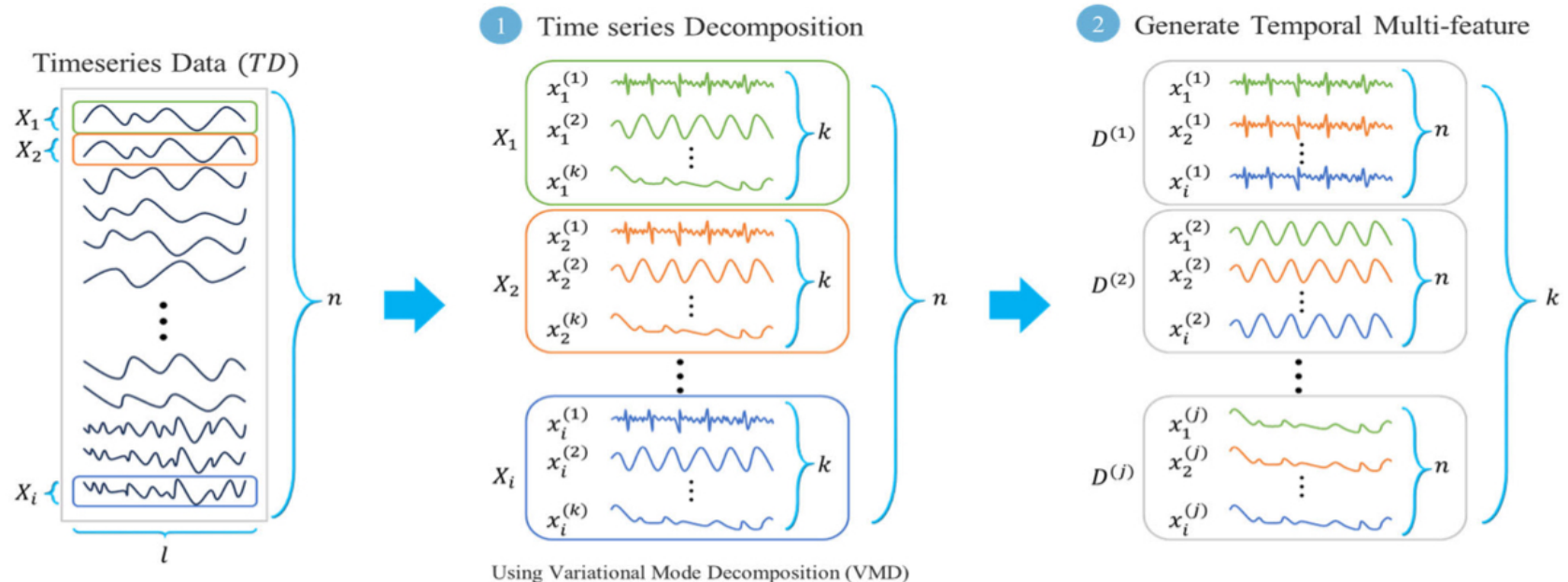
$$\min_{\{u_k\}, \{w_k\}} \left\{ \sum_k \left\| \partial_t \left[\left(\delta(t) + \frac{j}{\pi t} \right) * u_k(t) \right] e^{-jw_k t} \right\|_2^2 \right\}$$

$$s.t. \sum_{k=1}^K u_k(t) = V(t)$$

Temporal representation-learning-based clustering(TMRC)

■ Step 1) Generate temporal multiple-features

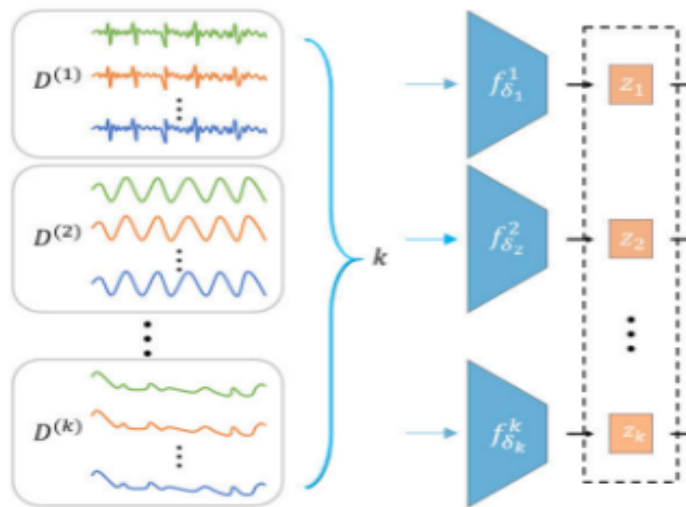
- 1-3) 각 부분 시계열(X_i)에서 분해된 k 개의 모드들을 동일한 인덱스(j)를 가진 성분끼리 결합하여 총 k 개의 독립적인 독립적인 ‘시간적 다중 특징 데이터셋($D^{(1)}, D^{(2)}, \dots, D^{(j)}$)’으로 재구성 ($1 \leq j \leq k$)
- $D^{(1)}$: n 개의 데이터에서 나온 “모드 1”의 집합(ex. 저주파 집합)
- $D^{(2)}$: n 개의 데이터에서 나온 “모드 2”의 집합(ex. 고주파 집합)
- $D^{(j)}$: n 개의 데이터에서 나온 “모드 j ”의 집합 (ex. 노이즈 집합)
- TMF = $\{D^{(1)}, D^{(2)}, D^{(3)}, \dots, D^{(k)}\}$ that consist of k temporal features for learning representation.



Temporal representation-learning-based clustering(TMRC)

■ Step 2) Temporal multiple-features representation learning(TMRL)

- [Encoder] 분해된 k 개의 모드에 대응하는 k 개의 독립적인 LSTM-Autoencoder 구축
 - $D^{(1)}$ 집합 $\rightarrow f_{\delta_1}^1$ (encoder network 1)
 - $D^{(2)}$ 집합 $\rightarrow f_{\delta_2}^2$ (encoder network 2)
 - $D^{(k)}$ 집합 $\rightarrow f_{\delta_k}^k$ (encoder network k)
- 긴 시계열 데이터에서 불필요한 정보는 덜어내고, 핵심적인 패턴만을 요약하여 추출하는 과정
- k 개의 모드를 하나의 LSTM-AE로 한번에 학습하는 것이 아니라, k 개의 별도의 LSTM-AE를 구축
- 서로 다른 주파수 특성을 가진 패턴들이 섞이지 않도록 학습을 진행



Input: VMD로 분해된 시계열 모드()

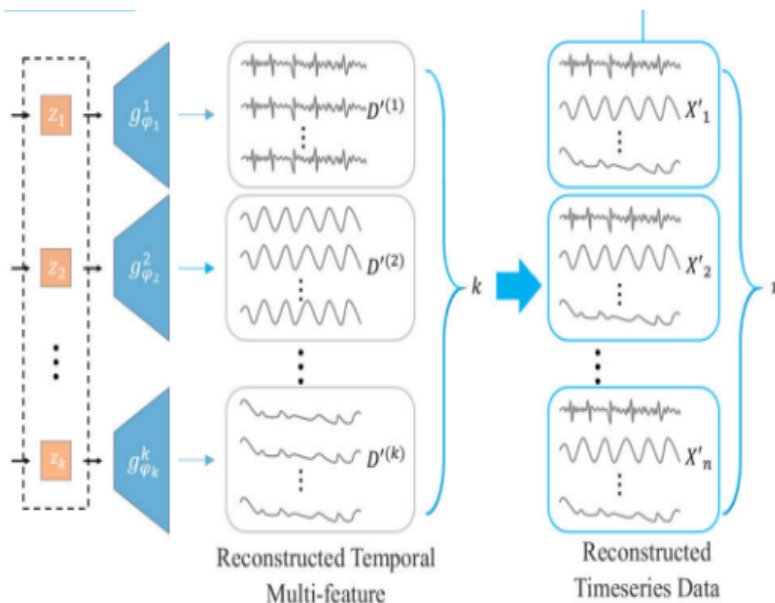
Process: LSTM 레이어를 통과하며 고차원 시계열 데이터를 저차원의 잠재 벡터(Latent Variable,)로 압축 (Three layer)

Output: 잠재 벡터(Latent Variable,)

Temporal representation-learning-based clustering(TMRC)

■ Step 2) Temporal multiple-features representation learning(TMRL)

- [Decoder] 분해된 k 개의 모드에 대응하는 k 개의 독립적인 LSTM-Autoencoder 구축
 - 잠재 벡터 $z_1 \rightarrow g_{\varphi_1}^1$ (decoder network 1)
 - 잠재 벡터 $z_2 \rightarrow g_{\varphi_2}^2$ (decoder network 2)
 - 잠재 벡터 $z_k \rightarrow g_{\varphi_k}^k$ (decoder network k)
- 추출된 특징이 원본의 정보를 얼마나 잘 담고 있는지 검증하는 단계
- 잠재 벡터 z 를 원본 데이터의 형태로 복원하는 과정으로, 원본으로 잘 복원이 된다면 인코딩 단계에서 핵심 정보를 잘 담아 축소된 것임



Input: 압축된 잠재 벡터()

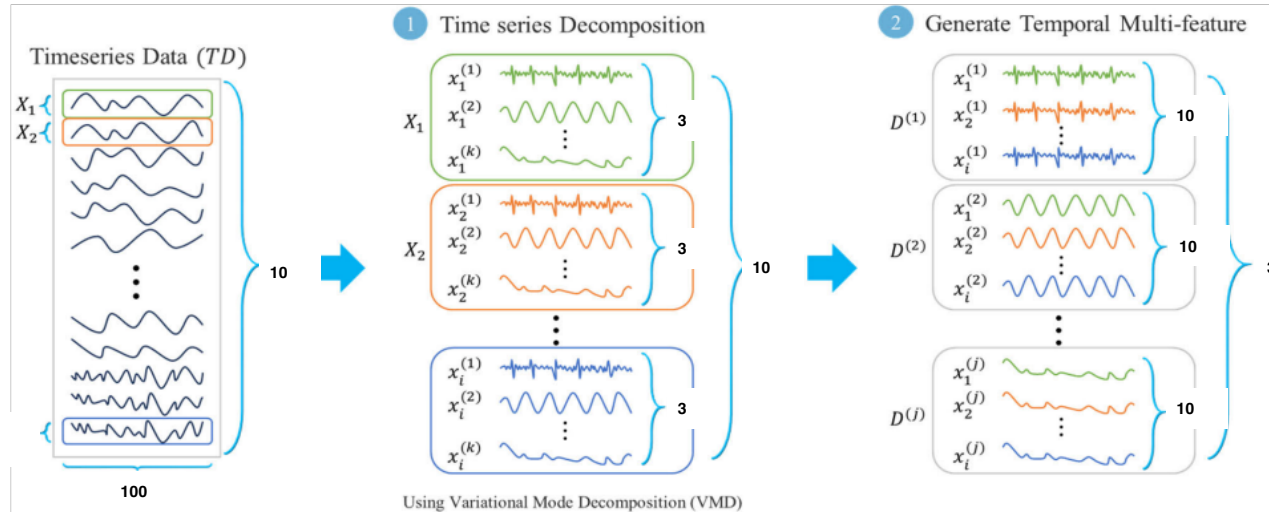
Process: 잠재 벡터를 다시 LSTM 레이어에 통과시켜 원본 시계열의 형태로 재구성(Reconstruction) () (Three layer)

Output: 재구성된 데이터(X')

Temporal representation-learning-based clustering(TMRC)

■ Step 2) Temporal multiple-features representation learning(TMRL)

- 예시: $\ell=100$, $n=10$, $k=3$, 은닉 차원=128



$$= \begin{bmatrix} X_1 \text{의 모드1:} & [0.1, 0.2, \dots, 0.9] \\ X_2 \text{의 모드1:} & [0.2, 0.3, \dots, 1.0] \\ \vdots & \vdots \\ X_{10} \text{의 모드1:} & [0.4, 0.5, \dots, 1.1] \end{bmatrix}$$

(10 x 100)

$$= \begin{bmatrix} X_1 \text{의 모드2:} & [-1.0, 1.0, \dots, -0.5] \\ \vdots & \vdots \\ X_{10} \text{의 모드2:} & [-0.5, 0.5, \dots, -0.1] \end{bmatrix}$$

(10 x 100)

$$= \begin{bmatrix} X_1 \text{의 모드3:} & [0.01, -0.01, \dots, 0.00] \\ \vdots & \vdots \\ X_{10} \text{의 모드3:} & [0.02, -0.02, \dots, 0.01] \end{bmatrix}$$

(10 x 100)

$$= \begin{bmatrix} z_{1,1}: & [2.5, 0.1, \dots, 1.1] & \leftarrow \text{총 128개 숫자} \\ z_{2,1}: & [2.6, 0.2, \dots, 1.2] \\ \vdots & \vdots \\ z_{10,1}: & [3.0, 0.9, \dots, 1.5] \end{bmatrix}$$

(10 x 128)

$$= \begin{bmatrix} z_{1,2}: & [0.5, -0.1, \dots, 0.3] & \leftarrow \text{총 128개 숫자} \\ \vdots & \vdots \\ z_{10,2}: & [0.6, -0.2, \dots, 0.4] \end{bmatrix}$$

(10 x 128)

$$= \begin{bmatrix} z_{1,3}: & [0.0, 0.1, \dots, 0.0] & \leftarrow \text{총 128개 숫자} \\ \vdots & \vdots \\ z_{10,3}: & [0.1, 0.0, \dots, 0.0] \end{bmatrix}$$

(10 x 128)

AE

양상블

$$Z = \begin{bmatrix} Z_1: & [z_{1,1} (128\text{개}) & z_{1,2} (128\text{개}) & z_{1,3} (128\text{개})] \\ Z_2: & [z_{2,1} (128\text{개}) & z_{2,2} (128\text{개}) & z_{2,3} (128\text{개})] \\ Z_3: & [z_{3,1} (128\text{개}) & z_{3,2} (128\text{개}) & z_{3,3} (128\text{개})] \\ \vdots & \vdots & \vdots & \vdots \\ Z_{10}: & [z_{10,1} (128\text{개}) & z_{10,2} (128\text{개}) & z_{10,3} (128\text{개})] \end{bmatrix}$$

같은 주파수 영역끼리 모여 3개의 행렬을 형성

AE를 통해 잠재 벡터 생성
특징 행렬 생성

3개의 특징 행렬을 이어 붙여
총 10개의 최종 통합 행렬 Z 생성

Temporal representation-learning-based clustering(TMRC)

■ Step 3) TMRC FRAMEWORK

- 앞서 완성된 통합 벡터 Z 가 클러스터링 알고리즘의 직접적인 입력으로 사용됨
 - Ex) $n=10$ 이라면 $Z = \{Z_1, Z_2, \dots, Z_{10}\}$
- 사용자가 정한 클러스터 개수 g 개만큼 무작위로 중심점 c 가 선택됨
 - $c_1, c_2, \dots, c_g \subset \{Z_1, Z_2, \dots, Z_{10}\}$
 - Ex) $g=2, c_1 = Z_1, c_2 = Z_{10}$

$$\sum_{j=1}^k \|z_j - c_n\|^2$$

$$\text{Dist}(Z_2, c_1) = \sqrt{(z_{2,1} - z_{1,1})^2 + (z_{2,2} - z_{1,2})^2 + \dots + (z_{2,384} - z_{1,384})^2}$$

$$\text{Dist}(Z_2, c_2) = \sqrt{(z_{2,1} - z_{10,1})^2 + (z_{2,2} - z_{10,2})^2 + \dots + (z_{2,384} - z_{10,384})^2}$$

- 임의로 설정된 초기 중심점(c)과 개별 벡터(Z)간 유클리드 거리 최소화 원칙에 기반하여 각 데이터를 클러스터에 할당
- 해당 과정을 반복하면서 클러스터의 중심을 새롭게 갱신하고, 중심점의 위치가 수렴하면(갱신이 멈추면) 최종 군집을 확정

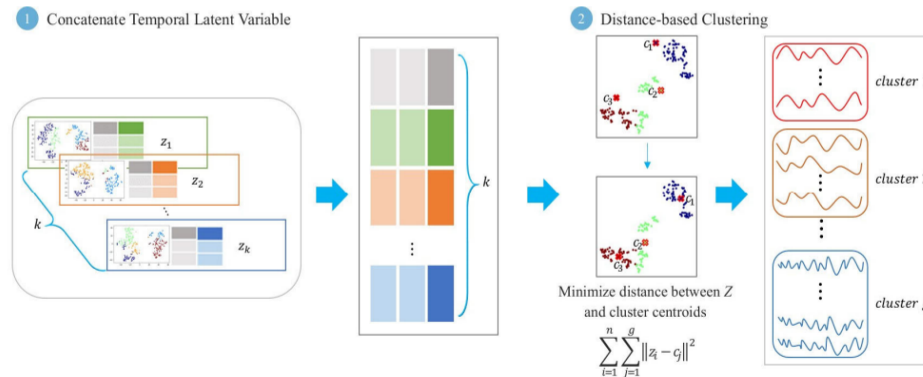


FIGURE 3. Procedure of temporal multi-features representation learning based clustering.

Temporal representation-learning-based clustering(TMRC)

■ Algorithm

Algorithm 1 TMRC

Input: Time-series data $TD = \{X_1, X_2, \dots, X_n\}$

Number of decomposition k ;

Set number of clusters g ;;

Output:

Clustering labels Q

// Step 1: Generate Temporal Multi-features

1: **For** 1 to n **do** // Decomposition

2: Decomposition each of time-series data TD by Eq. (1);

3: **End**

4: Combine each time-series decomposition data

5: **Return** Temporal Multi-feature

$TMF = \{D^{(1)}, D^{(2)}, \dots, D^{(k)}\}$

// Step 2: Temporal multiple-feature representation learning

5: **For** 1 to k **do** // Each k Autoencoder Training

6: $\delta_k, \varphi_k \leftarrow$ Initialize parameters

7: **Repeat**

8: Calculate sum of reconstruction error by Eq. (2);

9: $\delta_k, \varphi_k \leftarrow$ Update parameters using gradients of
 $L(\delta_k, \varphi_k)$

10: **until** convergence of parameters δ_k, φ_k

11: **End**

12: **Return** Temporal latent variable z

// Step 3: Clustering

13: Randomly select g initial cluster centers c .

14: **Repeat**

15: Calculate the distance between each Temporal latent
variable z and cluster centers c by Eq. (3);

16: Assign each Temporal Feature to the cluster which
has the closest centroid

17: **Until** no changing in the center of clusters.

18: **Return** Clustering labels Q

Input:

원본 시계열 데이터: TD

VMD 파라미터: k

클러스터 개수: g

Temporal representation-learning-based clustering(TMRC)

■ Experiments(I)

- UCR Archive에서 제공하는 36개의 공개 시계열 데이터 셋을 통해 TMRC의 성능 평가
 - Image, Spectro, Sensor, ECG, Motion, Simulated 형태의 시계열 데이터를 활용

- 클러스터링의 평가 지표로 RI와 NMI를 활용

- RI: 예측된 클러스터와 실제 클러스터 간의 일치 정도를 평가
- NMI: 예측된 클러스터와 실제 클러스터 레이블 간의 정보량 유사도를 측정

- Hyperparameters: Grid Search

- 입력 시계열 길이(ℓ): 10, 50, 100
- 시계열 분해 개수(k): 2, 3, 4, 5, 6
- 은닉 차원 크기: 128, 100, 400, 800, 1200
- 클러스터 개수 g 는 각 데이터셋에서 공개되어 있는 정답 class를 사용함 ex) TD1 Class=3 $\Rightarrow g=3$

Notation	Dataset Name	Train Size	Test Size	Length	Class	Dataset Type
TD1	Arrow	36	175	251	3	Image
TD2	Beef	30	30	470	5	Spectro
TD3	BeetleFly	20	20	512	2	Image
TD4	BirdChicken	20	20	512	2	Image
TD5	Car	60	60	577	4	Sensor
TD6	ChlorineConcentration	467	3840	166	3	Sensor
TD7	Coffee	28	28	286	2	Spectro
TD8	DiatomsizeReduction	16	306	345	4	Image
TD9	Dist.pha.outLagegroup	400	139	80	3	Image
TD10	Dist.pha.outL.correct	600	276	80	2	Image
TD11	ECG200	100	100	96	2	ECG
TD12	ECGFiveDays	23	861	136	2	ECG
TD13	GunPoint	50	150	150	2	Motion
TD14	Ham	109	105	431	2	Spectro
TD15	Herring	64	64	512	2	Image
TD16	Lighting2	60	61	637	2	Sensor
TD17	Meat	60	60	448	3	Spectro
TD18	Mid.pha.outLagegroup	400	154	80	3	Image
TD19	Mid.pha.outL.correct	600	291	80	2	Image
TD20	Mid.pha.TW	399	154	80	6	Image
TD21	MoteStrain	20	1252	84	2	Sensor
TD22	OSULeaf	200	242	427	6	Image
TD23	Plane	105	105	144	7	Sensor
TD24	Prox.pha.outLageGroup	400	205	80	3	Image
TD25	Prox.pha.TW	400	205	80	6	Image
TD26	SonyAIBORobotSurface	20	601	70	2	Sensor
TD27	SonyAIBORobotSurfaceII	27	953	65	2	Sensor
TD28	SwedishLeaf	500	625	128	15	Image
TD29	Symbols	25	995	398	6	Image
TD30	ToeSegmentation1	40	228	277	2	Motion
TD31	ToeSegmentation2	36	130	343	2	Motion
TD32	TwoLeadECG	1000	4000	128	4	ECG
TD33	TwoPatterns	23	1139	82	2	Simulated
TD34	Wafer	1000	6164	152	2	Sensor
TD35	Wine	57	54	234	2	Spectro
TD36	WordsSynonyms	267	638	270	25	Image

Temporal representation-learning-based clustering(TMRC)

■ Experiments(II)

- 36개의 데이터셋 중 12개의 데이터에서 RI 최고 성능, 13개의 데이터에서 NMI 최고 성능 달성

TABLE 3. Comparison results of the RI for EXP-I.

Data	UDFS	NDFS	RDFS	RSFS	KSC	KDBA	K-Shape	U-shape	DEC	IDEC	DTG	DTCR	STCN	TS2Vec	R-Clust	TCGAN	TMRC
TD1	0.7254	0.7381	0.7476	0.7108	0.7254	0.7222	0.7254	0.6460	0.5817	0.6210	0.6692	0.6868	0.7234	0.6445	0.6910	0.6541	0.9842
TD2	0.6759	0.7034	0.7149	0.6975	0.7057	0.6713	0.5402	0.6966	0.5954	0.6276	0.6345	0.8046	0.7471	0.5793	0.6701	0.6633	0.8229
TD3	0.4949	0.5579	0.6053	0.6516	0.6053	0.6052	0.6053	0.7134	0.4947	0.6053	0.5211	0.9000	1.0000	0.7038	0.5333	1.0000	
TD4	0.4947	0.7316	0.5579	0.6632	0.7316	0.6053	0.6632	0.5579	0.4737	0.4789	0.4947	0.8105	1.0000	0.6053	0.5500	0.4885	1.0000
TD5	0.6757	0.6260	0.6667	0.6708	0.6898	0.6254	0.7028	0.6418	0.6859	0.6870	0.6695	0.7501	0.7372	0.6463	0.7455	0.7057	0.7322
TD6	0.5282	0.5225	0.5330	0.5316	0.5256	0.5300	0.4111	0.5318	0.5348	0.5350	0.5353	0.5357	0.5384	0.4958	0.5082	0.5311	0.5372
TD7	0.8624	1.0000	0.5476	1.0000	1.0000	0.4851	1.0000	1.0000	0.4921	0.5767	0.4841	0.9286	1.0000	0.4841	0.7773	0.8649	1.0000
TD8	0.9583	0.9583	0.9333	0.9137	1.0000	0.9583	1.0000	0.7083	0.9294	0.7347	0.8792	0.9682	0.9921	0.9087	0.8100	0.7199	0.8539
TD9	0.6531	0.6239	0.6252	0.6539	0.6535	0.6750	0.6020	0.6273	0.7785	0.7786	0.7812	0.7825	0.7825	0.4870	0.7404	0.7212	0.6477
TD10	0.5362	0.5362	0.5252	0.5327	0.5235	0.5203	0.5252	0.5098	0.5029	0.5330	0.5010	0.6075	0.6277	0.5098	0.4999	0.4995	0.5722
TD11	0.6533	0.6315	0.7018	0.6916	0.6315	0.6018	0.7018	0.5758	0.6422	0.6233	0.6018	0.6648	0.7018	0.5404	0.6231	0.6181	0.7424
TD12	0.5020	0.5573	0.5020	0.5953	0.5257	0.5573	0.5020	0.5968	0.5103	0.5114	0.5016	0.9638	0.6504	0.6214	0.5123	0.5007	0.6492
TD13	0.5029	0.5102	0.6498	0.4994	0.4971	0.5420	0.6278	0.6278	0.4981	0.4974	0.5400	0.6398	0.7575	0.5039	0.4975	0.4975	0.6191
TD14	0.5219	0.5362	0.5107	0.5127	0.5362	0.5141	0.5311	0.5362	0.5963	0.4956	0.5648	0.5362	0.5879	0.5337	0.5170	0.5051	1.0000
TD15	0.5099	0.5164	0.5238	0.5151	0.4940	0.5164	0.4965	0.5417	0.5099	0.5099	0.5045	0.5759	0.6036	0.5099	0.4962	0.4980	0.5893
TD16	0.5119	0.5373	0.5729	0.5269	0.6263	0.5119	0.6548	0.5192	0.5311	0.5519	0.5770	0.5913	0.6786	0.5148	0.5110	0.5656	0.6590
TD17	0.6483	0.6635	0.6578	0.6657	0.6723	0.6816	0.6575	0.6742	0.6475	0.6220	0.3220	0.9763	0.9186	0.5695	0.8234	0.7324	0.8847
TD18	0.5269	0.5350	0.5315	0.5473	0.5364	0.5513	0.5105	0.5396	0.7059	0.6800	0.5757	0.7982	0.7975	0.4587	0.7325	0.7284	0.6003
TD19	0.5431	0.5047	0.5114	0.5149	0.5014	0.5563	0.5114	0.5218	0.5423	0.5423	0.5272	0.5617	0.5442	0.4996	0.4997	0.4997	0.6263
TD20	0.1225	0.1919	0.7920	0.8062	0.8187	0.8046	0.6213	0.7920	0.8590	0.8626	0.7115	0.8638	0.8625	0.6147	0.7846	0.7826	0.6850
TD21	0.5579	0.6053	0.5579	0.6168	0.6632	0.4789	0.6053	0.4789	0.7435	0.7324	0.5062	0.7686	0.6560	0.5001	0.7787	0.7580	0.7412
TD22	0.5372	0.5622	0.5497	0.5665	0.5714	0.5541	0.5538	0.5525	0.7484	0.7607	0.7329	0.7739	0.7615	0.7378	0.8266	0.7396	0.7644
TD23	0.8949	0.8954	0.9220	0.9314	0.9603	0.9225	0.9901	1.0000	0.9447	0.9447	0.9040	0.9549	0.9663	0.8368	0.9947	0.9489	0.9472
TD24	0.4997	0.5463	0.5780	0.5384	0.5305	0.5192	0.5617	0.5206	0.4263	0.8091	0.7430	0.8091	0.8379	0.4928	0.8001	0.7816	0.8189
TD25	0.4947	0.6053	0.5579	0.5211	0.6053	0.5211	0.5211	0.4789	0.8189	0.9030	0.8380	0.9023	0.8984	0.5669	0.7880	0.7877	0.7206
TD26	0.7695	0.7721	0.7787	0.7928	0.7726	0.7988	0.8084	0.7639	0.5732	0.6900	0.5563	0.8769	0.7356	0.5083	0.8878	0.5018	0.6529
TD27	0.4745	0.8865	0.8756	0.8948	0.9039	0.8684	0.5617	0.8770	0.6514	0.6572	0.7012	0.8354	0.7417	0.5085	0.7170	0.6459	0.7025
TD28	0.4923	0.5500	0.5192	0.5038	0.4923	0.5500	0.5333	0.6154	0.8837	0.8893	0.8871	0.9223	0.8872	0.8885	0.9434	0.8937	0.8537
TD29	0.8548	0.8562	0.8525	0.9060	0.8982	0.9774	0.8373	0.9603	0.8841	0.8857	0.9053	0.9168	0.9088	0.7161	0.9842	0.8999	0.9088
TD30	0.4921	0.5429	0.4968	0.5060	0.6143	0.6143	0.5873	0.4983	0.5017	0.5077	0.5659	0.8177	0.6323	0.5142	0.4982	1.0000	
TD31	0.5257	0.5968	0.5968	0.5826	0.5257	0.5573	0.5257	0.5020	0.4901	0.4991	0.5348	0.8286	0.8186	0.5106	0.6515	0.5043	0.9846
TD32	0.5495	0.6328	0.8246	0.5635	0.5464	0.5476	0.8246	0.5404	0.5007	0.5016	0.5116	0.7619	0.6566	0.9176	0.5037	0.5873	
TD33	0.8259	0.8530	0.8385	0.8588	0.8585	0.8446	0.8046	0.7757	0.6293	0.6338	0.6251	0.6984	0.9486	0.6250	0.7288	0.6309	0.6295
TD34	0.4925	0.5263	0.5263	0.4925	0.4925	0.4925	0.4925	0.4925	0.5679	0.5597	0.5324	0.7338	0.8433	0.5004	0.5355	0.5469	0.9082
TD35	0.4827	0.5123	0.5021	0.5033	0.5064	0.5001	0.5033	0.4913	0.5157	0.4906	0.6271	0.6925	0.5080	0.5047	0.4965	1.0000	
TD36	0.8697	0.8706	0.8861	0.8817	0.8727	0.8159	0.7844	0.8230	0.8983	0.8947	0.8855	0.8984	0.8748	0.8798	0.8991	0.9025	0.8875
Avg. Rank	11.5556	8.8056	9.0278	8.7778	8.7580	9.6389	9.6389	9.8333	10.6667	9.4444	10.9722	3.6944	3.3333	12.3056	7.9722	10.7778	4.8333
Avg. RI	0.6077	0.6402	0.6478	0.6542	0.6582	0.6335	0.6419	0.6402	0.6351	0.6515	0.6238	0.7114	0.7889	0.6052	0.6990	0.6486	0.7865
Num. Best	0	1	0	1	3	0	2	2	0	1	0	6	11	1	6	1	12
p-value	0.0000	0.0001	0.0004	0.0004	0.0008	0.0001	0.0002	0.0002	0.0002	0.0003	0.0000	0.8584	0.5128	0.0000	0.0321	0.0001	-

TABLE 4. Comparison results of the NMI for EXP-I.

Data	UDFS	NDFS	RDFS	RSFS	KSC	KDBA	K-Shape	U-shap	DEC	IDEC	DTC	DTCR	STCN	TS2Vec	R-Clust	TCGAN	TMRC
TD1	0.5240	0.4997	0.5975	0.5104	0.5240	0.4816	0.5240	0.3522	0.3100	0.2949	0.5000	0.5513	0.5240	0.2351	0.4012	0.2656	0.9538
TD2	0.2718	0.3647	0.3799	0.3597	0.3828	0.3340	0.3338	0.3413	0.2463	0.2463	0.2751	0.5473	0.5432	0.1874	0.2466	0.2530	0.7424
TD3	0.0371	0.1264	0.1919	0.2795	0.2215	0.2783	0.3456	0.5105	0.0308	0.0082	0.3456	0.7610	1.0000	1.0000	0.4636	0.0669	1.0000
TD4	0.0371	0.3988	0.1187	0.3002	0.3988	0.2167	0.3456	0.2783	0.0160	0.0082	0.0073	0.5310	1.0000	0.3437	0.2420	0.0020	1.0000
TD5	0.2319	0.2361	0.2511	0.2920	0.2719	0.2691	0.3771	0.3655	0.2766	0.2972	0.1892	0.5021	0.5701	0.2607	0.5582	0.2917	0.3917
TD6	0.0138	0.0075	0.0254	0.0159	0.0147	0.0164	0.0000	0.0135	0.0009	0.0008	0.0013	0.0195	0.0760	0.0091	0.0273	0.0029	0.0494
TD7	0.6945	1.0000	0.2513	1.0000	1.0000	0.0778	1.0000	1.0000	0.0120	0.1431	0.5523	0.6277	1.0000	0.0158	0.4703	0.6952	1.0000
TD8	0.9300	0.9300	0.8734	0.8761	1.0000	0.9300	1.0000	0.4849	0.8030	0.5140	0.6863	0.9418	1.0000	0.8251	0.6009	0.5421	0.6324
TD9	0.3262	0.1943	0.2762	0.3548	0.3331	0.4261	0.2911	0.2577	0.4405	0.4400	0.3406	0.4553	0.5037	0.1906	0.4223	0.3171	0.3298
TD10	0.0473	0.0567	0.1071	0.0782	0.0261	0.0199	0.0527	0.0063	0.0011	0.0150	0.0115	0.1180	0.2327	0.0014	0.0015	0.0003	0.1436
TD11	0.1854	0.1403	0.2668	0.2918	0.1403	0.1886	0.3682	0.1323	0.1885	0.2225	0.0918	0.3691	0.4316	0.0609	0.1609	0.1289	0.3763
TD12	0.0600	0.1296	0.0352	0.1760	0.0682	0.1983	0.0002	0.1498	0.0178	0.0223	0.0022	0.8056	0.3582	0.1844	0.0195	0.0018	0.2758
TD13	0.0220	0.0334	0.2405	0.0152	0.0126	0.1288	0.3653	0.3653	0.0020	0.0031	0.0194	0.4200	0.5537	0.0105	0.0000	0.0000	0.3268
TD14	0.0389	0.0595	0.0980	0.0256	0.0595	0.0265	0.0517	0.0619	0.1508	0.1285	0.1016	0.0989	0.2382	0.0730	0.0273	0.0111	1.0000
TD15	0.0253	0.0225	0.0518	0.0236	0.0027	0.0000	0.0027	0.1324	0.0306	0.0207	0.0143	0.2248	0.2002	0.0163	0.0000	0.0027	0.2395
TD16	0.0047	0.0851	0.1426	0.0326	0.1979	0.0850	0.2670	0.0144	0.0600	0.1248	0.1435	0.2289	0.3479	0.0440	0.0210	0.0849	0.3590
TD17	0.2832	0.2416	0.1943	0.3016	0.2846	0.3661	0.2254	0.2716	0.5176	0.2250	0.2250	0.9653	0.9393	0.1099	0.6264	0.5060	0.7980
TD18	0.1105	0.0416	0.1595	0.0968	0.1061	0.1148	0.0722	0.1491	0.2686	0.2199	0.1390	0.4661	0.5109	0.0473	0.3982	0.3865	0.1725
TD19	0.0713	0.0150	0.0443	0.0321	0.0053	0.0760	0.0349	0.0253	0.1005	0.0083	0.0079	0.1150	0.0921	0.0111	0.0010	0.0010	0.1840
TD20	0.4276	0.4149	0.5366	0.4219	0.4486	0.4497	0.5229	0.4065	0.4509	0.3444	0.1156	0.5503	0.6163	0.1106	0.4274	0.4046	0.4802
TD21	0.1187	0.1919	0.1264	0.2373	0.3002	0.0970	0.2215	0.0082	0.3867	0.3821	0.0094	0.4094	0.4609	0.0008	0.4883	0.4487	0.3889
TD22	0.0200	0.0352	0.0246	0.0463	0.0421	0.0327	0.1206	0.0203	0.2141	0.2412	0.2021	0.2599	0.3344	0.1889	0.4982	0.1834	0.3012
TD23	0.0846	0.8414	0.8675	0.8736	0.9218	0.8784	0.9642	1.0000	0.8947	0.8947	0.8678	0.9296	0.9515	0.6399	0.9819	0.8949	0.8917
TD24	0.0182	0.0380	0.0726	0.0938	0.0682	0.0377	0.0110	0.0332	0.2500	0.5396	0.4153	0.5581	0.6317	0.1481	0.5638	0.4909	0.5721
TD25	0.0308	0.2215	0.1178	0.0809	0.1919	0.2167	0.1577	0.0107	0.5864	0.3289	0.6199	0.6539	0.7330	0.0721	0.5455	0.5094	0.5032
TD26	0.6122	0.6112	0.6278	0.6368	0.6129	0.5516	0.7107	0.5803	0.2773	0.4451	0.2559	0.6634	0.6112	0.0004	0.6724	0.0171	0.2300
TD27	0.4802	0.5413	0.5107	0.5406	0.5619	0.5481	0.0110	0.5903	0.2214	0.2327	0.4257	0.6121	0.5647	0.0002	0.3485	0.2103	0.3745
TD28	0.0082	0.0394	0.0457	0.0269	0.0743	0.1271	0.1041	0.3456	0.5569	0.5573	0.6187	0.6663	0.6106	0.4009	0.7596	0.5574	0.3909
TD29	0.0727	0.7593	0.7174	0.8027	0.8264	0.9388	0.6366	0.8691	0.7421	0.7419	0.7995	0.8989	0.8949	0.1091	0.9240	0.7982	0.8159
TD30	0.0089	0.2141	0.0880	0.0174	0.0202	0.2712	0.3073	0.3073	0.0010	0.0010	0.0188	0.3315	0.3671	0.2294	0.0223	0.0001	1.0000
TD31	0.0727	0.1713	0.1713	0.1625	0.0863	0.2627	0.0863	0.1519	0.0065	0.0118	0.0096	0.3249	0.5498	0.0004	0.1453	0.0418	0.9200
TD32	0.3393	0.4351	0.4678	0.4068	0.4705	0.4419	0.3949	0.2979	0.0195	0.0142	0.0119	0.4713	0.4010	0.1376	0.7444	0.0061	0.2887
TD33	0.0004	0.1353	0.1238	0.0829	0.0011	0.0103	0.0000	0.0529	0.0017	0.0010	0.0036	0.4614	0.6911	0.1051	0.3255	0.0235	0.1347
TD34	0.0010	0.0546	0.0746	0.0194	0.0010	0.0000	0.0010	0.0101	0.0165	0.0108	0.0008	0.0228	0.2089	0.0819	0.0002	0.0000	0.5019
TD35	0.0045	0.0259	0.0065	0.0096	0.0094	0.0211	0.0119	0.0171	0.0018	0.1708	0.0000	0.2580	0.5927	0.0283	0.0130	0.0011	1.0000
TD36	0.4745	0.5396	0.5623	0.5462	0.4874	0.4527	0.4154	0.3933	0.4134	0.387	0.3498	0.5448	0.3947	0.2761	0.4659	0.4812	0.4210
Ave. RMI	11.4167	9.1944	8.7222	8.6667	8.7500	9.3333	9.1667	9.9464	10.6389	10.8611	11.5833	13.189	2.6667	12.4167	8.1667	12.1111	4.6111
Ave. NMI	0.2240	0.2764	0.2624	0.2812	0.2808	0.2659	0.2841	0.2777	0.2365	0.2308	0.2332	0.4818	0.5478	0.1735	0.3509	0.2397	0.5291
Num. Best	0	1	1	1	2	0	3	2	0	0	0	3	15	1	5	0	13
p-value	0.0000	0.0002	0.0001	0.0003	0.0004	0.0001	0.0002	0.0001	0.0000	0.0000	0.0000	0.8706	0.2419	0.0000	0.0249	0.0000	-

Temporal representation-learning-based clustering(TMRC)

■ 한계점 분석 및 제언

• 1. VMD의 분해 성능에 의존

- 만약 VMD가 노이즈를 유의미한 모드로 오인하여 분해하거나, 복잡한 비선형 패턴을 완벽히 분리하지 못할 경우, 이후 단계인 LSTM-Autoencoder는 왜곡된 특징을 그대로 학습하게 되어 최종 클러스터링 성능의 저하를 초래할 수 있음

• 2. 분해 모드의 개수(k) 결정 방식의 한계

- 최적의 k 를 선정하기 위해 RI, NMI와 같은 정답 레이블 기반의 평가지표를 활용하여 그리드 서치를 수행함
- 클러스터링의 본질이 정답을 모르는 비지도 학습임을 고려할 때, 이러한 방식으로 최적의 k 를 결정하는 것은 불가능하다는 제약이 있음
 - ✓ 순열 엔트로피 (Permutation Entropy, PE)를 통해 VMD의 분해 성능을 검증하고 최적의 k 를 찾도록 개선
 - ✓ 직교성 지표 (Index of Orthogonality, IO)를 통해 분해된 모드들 간 내적을 계산해 모드끼리 서로 독립적인지 평가
 - ✓ PE를 통해 최적의 k 를 선정하고, 그 결과가 타당한지 IO로 교차 검증을 제안

• 3. 특징 추출(Step 1,2)과 클러스터링(Step 3)의 분리

- 클러스터링 단계에서 발생하는 오류나 정보가 특징 추출 네트워크로 역전파 되지 않는 비종단간(Non-end-to-end) 학습 구조임
- K-Means 알고리즘이 수행되는 동안 특징 추출 모델의 파라미터를 동적으로 업데이트하여 클러스터링 성능을 직접적으로 향상시키기 위해 특징 공간을 미세 조정할 수 없음

• 4. LSTM-AE + Attention Mechanism

- 성능 향상을 기대

The proposed model has several limitations. First, the learning representations are not fully shared across features within time-series data. Second, the model uses K-Means for clustering. However, these limitations may be improved. In the future, these

Deep time-series clustering via latent representation alignment (DEETO)

■ DEETO

- 위상적 개념 도입
 - 시계열 데이터에서 '위상'이란 단순한 데이터 포인트의 좌표가 아닌, 시간의 흐름에 따른 데이터 간의 연결성과 구조적 분포
 - 분해 없이 잠재 공간(Latent Space)에서 데이터들이 형성하는 고유한 구조를 파악하여 클러스터링을 극대화
 - TMRC에서는 복잡한 시계열 데이터를 분석하기 위해 VMD를 통해 데이터를 분해(Decomposition)하는 방식을 택함
 - DEETO의 경우 데이터를 분해하는 대신 데이터가 뭉치는 위상학적 원리(Topology) 자체를 모델에게 학습시키는 방식으로 클러스터링을 진행함
- Key Mechanism
 - Eigenvector-based Prototypes (고유벡터 기반 프로토타입)
 - ✓ 클러스터 중심 근처의 샘플들로 그래프 행렬을 구성하고, 고유값 분해를 수행하여 데이터의 분산을 가장 잘 설명하는 첫 번째 고유벡터를 프로토타입으로 정의
 - Topological Loss Function (위상학적 손실 함수)
 - ✓ 개별 데이터(z_i)를 클러스터 프로토타입에 맞춰 정렬
- Pretraining Phase + Fine-tuning Phase

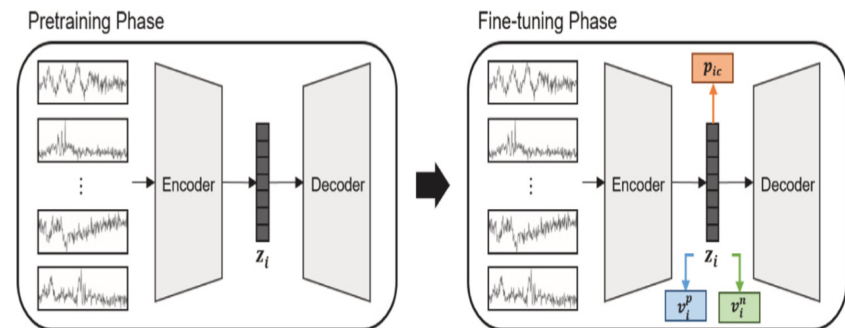


Fig. 1. Overview of the proposed method.

Deep time-series clustering via latent representation alignment (DEETO)

■ Pretraining Phase

- MLP 기반의 오토인코더 사용
- 원본 시계열 데이터(x)를 인코더에 통과시켜 압축된 잠재 표현(z)을 만들고, 이를 디코더를 통해 원본으로 복원
- Loss Function을 통해 원본 데이터와 복원된 데이터와의 차이를 최소화

$$\mathcal{L}_{RE} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2$$

- Mean Squared Error (MSE)를 최소화하는 방향으로 학습이 진행

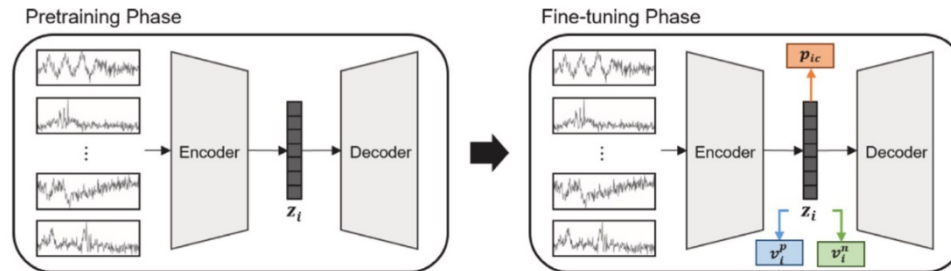


Fig. 1. Overview of the proposed method.

- Input: 원본 시계열 데이터 $D = \{x_i\}$ ($i=1 \sim N$)
- Process: Enc-Dec를 반복하며 오차가 최소화된 잠재 표현 z_i 생성
- Output: 잠재 표현 $\{z_i\}$ ($i=1 \sim N$)

Deep time-series clustering via latent representation alignment (DEETO)

■ Fine-tuning Phase

- 초기 클러스터 중심(μ_c) 설정
 - 앞 단계에서 만들어진 잠재 표현 집합 $\{z_1, z_2, z_3, \dots, z_N\}$ ($i=1 \sim N$)에 대해 K-Means Clustering을 수행
 - 이를 통해 C개의 초기 클러스터 중심(Cluster Centers, μ_c)을 도출
 - 이렇게 만들어진 클러스터 중심(μ_c)은 이후 단계에서 계산할 클러스터링 손실과 위상학적 손실을 계산하기 위한 기준점이 됨
- 클러스터링 손실(L_{CL})
 - 흩어져 있는 잠재 표현(z_i)들이 클러스터 중심 주변(μ_c)으로 밀집되도록 학습
 - 모델이 스스로 만든 예측(P)을 더 확실하게 다듬어서(Q) 그것을 정답처럼 여기고 학습하는 자기 학습 방식
 - 1. 모델은 현재 잠재 표현이 어느 클러스터에 속할지 확률(p_{ic})을 계산
 - ✓ 이때 스튜던트 t-분포를 활용하여 멀리 떨어진 데이터도 포용 → 이상치에 강함

- 2. 계산된 p_{ic} 을 제공하여 확률을 강화함
 - ✓ 이렇게 만들어진 q_{ic} 는 높은 확률은 더 높게, 낮은 확률은 더 낮게 만들어 “더 확실한 정답지” 역할을 수행

- 3. 모델은 자신의 예측(P)과 이 목표(Q) 사이의 차이(KL Divergence)를 계산
 - ✓ 이 손실 함수는 z 들이 클러스터 중심으로 모이도록 잠재 공간 $p_{ic} = \frac{(1 + \|z_i - \mu_c\|^2)^{-1}}{\sum_{c=1}^C (1 + \|z_i - \mu_c\|^2)^{-1}}$.

$$q_{ic} = \frac{p_{ic}^2 / \sum_{i=1}^N p_{ic}}{\sum_{c=1}^C (p_{ic}^2 / \sum_{i=1}^N p_{ic})}.$$

$$\mathcal{L}_{CL} = KL(Q \parallel P) = \sum_{i=1}^N \sum_{c=1}^C q_{ic} \log \frac{q_{ic}}{p_{ic}},$$

Deep time-series clustering via latent representation alignment (DEETO)

■ Fine-tuning Phase

- 위상학적 손실(L_{TP})

- 앞서 만들어진 L_{RE} 와 L_{CL} 은 시계열의 연속성(연결성)을 모두 반영하지 못한다는 한계가 존재
- 위상학적 손실은 해당 논문의 핵심적인 파트로 각 클러스터의 '대표적인 시간적 구조'를 수학적으로 찾아내는 과정

- 1. 각 클러스터 중심(μ_c)과 가장 가까운 M개의 샘플 선별

- ✓ K-Means로 나눈 클러스터 안에는 잘못 들어온 데이터가 섞여 있을 수 있어서 → 노이즈 배제
- ✓ 가깝다의 기준은 중심점(μ_c)과 개별 데이터(z) 사이의 유클리드 거리 기준

- 2. 선별된 샘플(M)들로 그래프 행렬 구성

- ✓ 중심점(μ_c)을 이용해 기초 행렬을 만들고, 선별된 M개의 샘플을 하나씩 가져와서 각 데이터가 가진 방향성 정보를 행렬에 더해줌
- ✓ 만들어진 최종 그래프 행렬은 클러스터 내부의 핵심 데이터들이 공통적으로 가리키고 있는 방향과 구조를 응축하고 있음

$$\delta_i = \|z_i - \mu_c\|,$$

- 3. Eigendecomposition: 고유값 분해를 통해 첫 번째 고유 벡터 도출

- ✓ 그래프 행렬을 분해하면 여러 개의 고유벡터와 고유값이 나오
- ✓ 여기서 가장 큰 고유값을 가진 첫 번째 고유벡터 $\Sigma_c^0 = \mu_c \mu_c^T$, 값을 가진 $\Sigma_c = \Sigma_c^0 + \sum_{z_j \in Z_c} z_j z_j^T$. 이나 방향을 의미

- 4. Result: v_c = 해당 클러스터의 프로토타입

- ✓ 클러스터의 위상학적 정보를 반영함

Deep time-series clustering via latent representation alignment (DEETO)

■ Fine-tuning Phase

- 위상학적 손실(L_{TP})
 - 앞서 만들어진 L_{RE} 와 L_{CL} 은 시계열의 연속성(연결성)을 모두 반영하지 못한다는 한계가 존재
 - 위상학적 손실은 해당 논문의 핵심적인 파트로 각 클러스터의 '대표적인 시간적 구조'를 수학적으로 찾아내는 과정
- 5. 유사도 측정 및 타겟 설정
 - ✓ 개별 데이터와 프로토타입 간의 코사인 유사도를 계산
 - ✓ Positive Prototype (v_i^p): 유사도가 가장 높은 클러스터의 프로토 타입
 - ✓ Negative Prototype (v_i^n): 두 번째로 유사도가 높은 클러스터의 프로토타입

$$\zeta_{ic} = \frac{\mathbf{z}_i \cdot \mathbf{v}_c}{\|\mathbf{z}_i\| \|\mathbf{v}_c\|} \in [-1, 1]$$

- 6. 위상학적 손실 함수 계산
 - ✓ 각도 기반 유사도를 사용하여 정렬을 최적화 → 0(완전 나뉨) ~ 1(완전 같음)
 - ✓ 분자: Positive와의 각도가 0에 가까울수록 유사도 최대화
 - ✓ 분모: Negative와의 각도가 0에 가까울수록 유사도 최소화
 - ✓ Positive와는 당기고, Negative와는 밀어내는 과정을 반복하며 오차를 최소화

$$\mathcal{L}_{TP} = - \sum_{i=1}^N \frac{1 - (\cos^{-1}(\mathbf{z}_i \cdot \mathbf{v}_i^p / \|\mathbf{z}_i\| \|\mathbf{v}_i^p\|) / \pi)}{1 - (\cos^{-1}(\mathbf{z}_i \cdot \mathbf{v}_i^n / \|\mathbf{z}_i\| \|\mathbf{v}_i^n\|) / \pi)}$$

Deep time-series clustering via latent representation alignment (DEETO)

■ Optimization

- 모든 요소를 통합하여 동시에 최적화

$$L = L_{RE} + \lambda_1 L_{CL} + \lambda_2 L_{TP}$$

- 1. L_{RE} (Reconstruction)
- 2. L_{CL} (Clustering): 데이터를 클러스터 중심으로 밀집시킴
- 3. L_{TP} (Topological): 시계열 특성 반영, 데이터의 위상학적 구조에 따른 정렬 강화
- 4. λ_1, λ_2 : 가중치
- STEP 1. 원본 데이터(x)를 넣고 MLP 인코더로 학습
 - L_{RE} 만 사용됨. 데이터의 특징을 어느 정도 담고 있는 초기 잠재 변수 z 생성
- STEP 2. 앞 단계에서 만든 z 를 가지고 K-Means를 한 번 실행
 - 초기 클러스터 중심(μ_c)를 만들기 위해서
- STEP 3. 클러스터 중심이 잡혔으니 3개의 손실 함수가 계산 되면서 MLP 파라미터가 조정됨
 - 이 과정에서 z 값 자체가 변하고, 클러스터링이 완성됨
- 결과적으로 오토인코더 학습이 완료되는 동시에 클러스터링도 동시에 완료가 됨
 - 학습 과정 자체가 데이터를 클러스터 중심으로 이동시키는 과정이었기 때문에, 학습이 끝난 후 별도로 K-Means를 수행할 필요 없이 모델이 출력하는 확률값만으로 즉시 그룹을 확인할 수 있음

Deep time-series clustering via latent representation alignment (DEETO)

■ Algorithm

Algorithm 1 Prototype set

Input: time-series dataset $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, number of nearest samples for deriving prototype M , pretrained encoder f , number of clusters C , and clusters' center μ_c

Output: prototype set S

- 1: $S \leftarrow \emptyset$
 - 2: $\mathbf{z}_i \leftarrow f(\mathbf{x}_i)$ for $\mathbf{x}_i \in D$.
 - 3: **for** $c \in \{1, \dots, C\}$ **do**
 - 4: Initialize Gram matrix $\Sigma_c^0 \leftarrow \mu_c \mu_c^\top$.
 - 5: $\delta_i \leftarrow \|\mathbf{z}_i - \mu_c\|$ for $i = 1, \dots, N$.
 - 6: $Z_c \leftarrow \mathbf{z}_i$ s corresponding to M -smallest δ_i s.
 - 7: Update Gram matrix $\Sigma_c \leftarrow \Sigma_c^0 + \mathbf{z}_j \mathbf{z}_j^\top$ for $\mathbf{z}_j \in Z_c$.
 - 8: $\Lambda_c, V_c \leftarrow$ eigendecomposition of Σ_c .
 - 9: $\mathbf{v}_c \leftarrow$ 1st column of V_c .
 - 10: $S \leftarrow S \cup \{\mathbf{v}_c\}$
 - 11: **end for**
-

Algorithm 2 Topological loss function

Input: time-series dataset $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, pretrained encoder f , prototype set S , and number of clusters C

Output: topological loss $\mathcal{L}_{TP}$

- 1: **for** $i = 1, 2, \dots, N$ **do**
 - 2: $\mathbf{z}_i \leftarrow f(\mathbf{x}_i)$.
 - 3: Compute ζ_{ic} by Eq. (8) for $\mathbf{v}_c \in S$.
 - 4: $\mathbf{v}_i^p \leftarrow \mathbf{v}_c$ corresponding to 1st largest ζ_{ic} .
 - 5: $\mathbf{v}_i^n \leftarrow \mathbf{v}_{c'}$ corresponding to 2nd largest ζ_{ic} .
 - 6: **end for**
 - 7: Calculate $\mathcal{L}_{TP}$ by Eq. (9).
-

Algorithm 3 Learning procedure of DEETO

Input: dataset $D = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, pretrained encoder f , number of clusters C , update interval U , stopping threshold τ , and maximum epochs T

Output: updated f

- 1: Pretrain f .
 - 2: $\mathbf{z}_i \leftarrow f(\mathbf{x}_i)$ for $i = 1, 2, \dots, N$.
 - 3: Initialize cluster assignments $\mathbf{y}$ using K-means.
 - 4: Obtain cluster center μ_c for $c = 1, 2, \dots, C$.
 - 5: **while** $t \leq T$ **do**
 - 6: $\mathbf{z}_i \leftarrow f(\mathbf{x}_i)$ for $i = 1, 2, \dots, N$.
 - 7: Update prototype set S by Algorithm 1.
 - 8: **if** $t \% U == 0$ **then**
 - 9: $\mathbf{y}' \leftarrow \mathbf{y}$
 - 10: Update target distribution $\mathcal{Q}$ using Eqs. (3) and (4).
 - 11: $\mathbf{y} \leftarrow \arg\max_c p_{ic}$
 - 12: **if** $\text{sum}(\mathbf{y}' \neq \mathbf{y})/N < \tau$ **then**
 - 13: Stop training.
 - 14: **end if**
 - 15: **end if**
 - 16: $\mathcal{L}_{RE} \leftarrow$ Eq. (1), $\mathcal{L}_{CL} \leftarrow$ Eq. (2), $\mathcal{L}_{TP} \leftarrow$ Algorithm 2.
 - 17: Update f by Eq. (10).
 - 18: **end while**
-

Deep time-series clustering via latent representation alignment (DEETO)

■ Experiments

- UEA & UCR Time-series Repository의 15개 데이터셋 사용, 평가 지표: NMI, ADR
- Baselines (9 Methods):
 - Deep Clustering: DEC, IDEC, DEPICT, SDCN, EDESC, SACC
 - Deep Time-series Clustering (DTSC): DTC, DTCR
 - 실험 결과 15개 데이터셋 중 10개에서 NMI 최고/차석 성능을 기록함
 - ✓ RNN을 사용하는 DTCR보다 단순 MLP 구조인 DEETO가 더 우수한 성능을 보임

Table 2

NMI scores of 15 time-series datasets for all methods. The value in parentheses denotes the standard deviation in the corresponding dataset. For each dataset, the best NMI score is highlighted in boldface, and the second-best value is highlighted in underline.

Dataset	AE	DEC	IDEC	DEPICT	SDCN	DTC		EDESC	SACC	
AH	0.2424 (0.0094)	0.2450 (0.0284)	0.2513 (0.0229)	0.2358 (0.0544)	<u>0.2609</u> (0.0324)	0.2545 (0.0396)	<u>0.1070</u> (0.0575)	0.1187 (0.0893)	0.1212 (0.1140)	0.2629 (0.0120)
BC	0.0082 (0.0015)	0.0074 (0.0044)	0.0093 (0.0018)	0.0507 (0.0596)	0.0087 (0.0018)	0.0000 (0.0000)	0.0482 (0.0915)	0.0489 (0.0902)	0.2631 (0.2289)	<u>0.0693</u> (0.0715)
CA	0.2536 (0.0109)	0.1645 (0.0498)	0.2260 (0.0367)	0.1850 (0.1164)	0.2463 (0.0231)	0.2675 (0.0230)	0.2470 (0.0747)	0.1453 (0.0858)	0.1791 (0.0477)	<u>0.2578</u> (0.0247)
DO	0.2025 (0.0139)	0.2278 (0.0085)	0.2504 (0.0538)	0.1020 (0.0812)	0.2792 (0.0545)	0.2441 (0.0170)	0.2453 (0.0113)	0.1269 (0.0869)	0.3152 (0.0607)	<u>0.2999</u> (0.0431)
EC	<u>0.1759</u> (0.0337)	0.1964 (0.0952)	0.1536 (0.0381)	0.0930 (0.0656)	0.1755 (0.0129)	0.1225 (0.0069)	0.0910 (0.0282)	0.1181 (0.1136)	0.0915 (0.0411)	0.1669 (0.0574)
FA	0.3662 (0.0127)	0.3545 (0.0083)	0.3625 (0.0197)	0.1955 (0.0387)	0.3634 (0.0087)	0.4313 (0.0508)	<u>0.3716</u> (0.0216)	0.2862 (0.0180)	0.3705 (0.0196)	0.3668 (0.0245)
FI	0.2438 (0.0354)	0.1768 (0.0244)	0.2205 (0.0432)	0.1272 (0.0509)	0.2097 (0.0203)	<u>0.2381</u> (0.0388)	0.2166 (0.0466)	0.1645 (0.0622)	0.0767 (0.1219)	0.2378 (0.0667)
MO	0.0761 (0.0479)	0.1123 (0.0044)	<u>0.1215</u> (0.0104)	0.0385 (0.0430)	0.0985 (0.0222)	0.1104 (0.0056)	0.1082 (0.0055)	0.0880 (0.0547)	0.1254 (0.0102)	0.1204 (0.0127)
MS	0.2808 (0.0327)	0.2921 (0.0711)	0.3111 (0.0428)	0.2313 (0.1922)	0.2772 (0.0403)	0.2550 (0.0457)	0.1006 (0.0532)	0.2037 (0.1410)	0.3082 (0.0636)	<u>0.3052</u> (0.0497)
PC	0.2926 (0.0271)	0.4153 (0.1811)	<u>0.4940</u> (0.1714)	0.1048 (0.2042)	0.4813 (0.1337)	0.0502 (0.0602)	0.0084 (0.0107)	0.0516 (0.0588)	0.1956 (0.0548)	0.5327 (0.1623)
PO	0.4858 (0.0129)	0.5380 (0.0273)	0.4927 (0.0109)	0.2566 (0.2470)	0.5036 (0.0307)	0.5085 (0.0186)	0.5054 (0.0013)	0.0939 (0.1373)	0.4729 (0.0182)	<u>0.5086</u> (0.0147)
SL	0.5411 (0.0144)	0.5155 (0.0195)	0.5521 (0.0530)	0.3086 (0.0126)	0.5299 (0.0544)	0.4858 (0.0284)	0.5482 (0.0395)	0.4879 (0.0254)	<u>0.5685</u> (0.0390)	0.5720 (0.0320)
SY	<u>0.7524</u> (0.0308)	0.6999 (0.0487)	0.7290 (0.0697)	0.6951 (0.0358)	0.7273 (0.0741)	0.7219 (0.0508)	0.7497 (0.0567)	0.6647 (0.0699)	0.6739 (0.0535)	0.7589 (0.0142)
TR	0.5320 (0.0027)	0.5232 (0.0248)	0.5139 (0.0469)	0.4594 (0.0870)	0.5143 (0.0434)	0.5314 (0.0027)	0.9606 (0.0881)	0.4493 (0.1284)	0.3595 (0.1872)	<u>0.5322</u> (0.0022)
WS	0.4522 (0.0099)	0.4274 (0.0085)	0.4353 (0.0183)	0.2472 (0.0258)	0.4391 (0.0116)	<u>0.4473</u> (0.0182)	0.3621 (0.0304)	0.3833 (0.0128)	0.1845 (0.0227)	0.4399 (0.0098)
Avg. Rank	4.6000*	5.5333**	4.4000**	8.1333**	4.8667*	4.8667*	5.7222**	8.2667**	6.4000*	10.0000**
# of Top 1	2	2	1	0	0	2	0	0	3	0
# of Top 2	4	2	3	0	1	4	0	4	10	0

Comparison

■ TMRC

- 시계열 데이터를 VMD를 통해 저주파, 고주파 등 성분별로 쪼개고, 각각을 LSTM-AE를 통해 독립적으로 학습
- 이 방식은 주파수 대역에 어떤 패턴이 존재하는지 명확히 볼 수 있어 물리적 해석력이 높다는 장점이 있음
- 다만, 주파수 대역만큼 모델을 여러 개 만들어야 해서 연산량이 많음
- VMD 파라미터를 어떻게 설정하느냐에 따라 성능이 달라질 수 있다는 문제가 존재

■ DEETO

- End-to-End 방식으로 복원, 클러스터링, 위상 정렬이라는 세 가지 목표를 하나의 네트워크에서 최적화
- 오토인코더 내부에 3가지 손실 함수를 추가하여 모델이 스스로 데이터의 구조를 파악하고 정렬하도록 유도
- 일반적인 딥 클러스터링 모델보다 학습시간이 김
 - 위상학적 손실을 계산하는 과정에서 매 반복마다 각 클러스터의 그래프 행렬을 만들고, 고유값 분해 등을 수행하기에 학습 시간이 길다는 단점이 존재

Q&A